

AI 协作日志

一、项目背景

- 1、任务：复现论文《A Large-Scale Annotated Multivariate Time Series Aviation Maintenance Dataset from the NGAFID》
- 2、我的目标：完成维护前 vs 维护后二分类复现，并尝试一项合理改进
- 3、AI 工具：Gemini（核心逻辑与排错）、Chatgpt(概念解释)
- 4、人机协作原则：AI 定位为“知识检索器”与“代码原型生成器”。所有底层逻辑、数据流向验证、解决硬件瓶颈的最终方案均由学生本人主导并决定。

二、人机协同过程

2.1 论文理解与目标拆解阶段

Prompt: 你现在的角色是机器学习研究员+论文复现工程师，“请帮我梳理《A Large-Scale Annotated Multivariate Time Series Aviation Maintenance Dataset from the NGAFID》这篇论文的核心贡献、数据集层级结构以及它所采用的 Baseline 模型是什么？”

AI 提供: 总结了论文摘要，列出了基准模型（如 InceptionTime 和 MINIROCKET），并简单翻译了数据集介绍。

学生贡献（主导修正）: 发现 AI 生成的总结过于宏观，未厘清数据的映射关系。我亲自精读原论文后，明确了复现的核心难点在于“高维多元时间序列的处理”，并确立了以 flight_header.csv 为核心连接桥梁的数据打通方案。

2.2 数据处理阶段

（1）深入理解时序数据特性

学生贡献: 未过度依赖 AI，根据以往处理无人机飞行日志经验，独立理清了数据的物理映射逻辑——确认 flight_header.csv 中的 Master Index 列是核心枢纽。它通过索引号将“飞行元数据”（故障类型、航班号）与 one_parq 或 flight_data.pkl 里的“传感器时间序列数据”（转速、油压、电压等）实现了一一对应，为后续构建 Dataset 奠定了基础。

(2) Google Drive 开源下载链接失效

发现问题：原仓库提供的 Google Drive 数据集下载链接失效，代码在前期数据拉取阶段直接报错 (ReadError/NameError)。

学生贡献（主动解决）：绕过代码自动下载的局限，我手动将 Kaggle/Zenodo 上的源数据集文件下载并上传至我个人的 Google Drive 中，随后重构了 Dataset 挂载路径与文件读取链接，确保数据管道畅通。

2.3 模型复现阶段

(1) 多文件项目架构解析

Prompt：“GitHub 仓库提供了三个 .ipynb 文件 (DASK, TF, MINIROCKET)，它们各自的作用和适用场景是什么？我应该从哪个入手？”

AI 提供：分析了三个文件的定位 (Dask 用于海量 Parquet 探索；TF 用于深度学习如 InceptionTime/ConvMHSA；MiniRocket 用于快速特征提取)。

学生贡献：基于 AI 的分类，我制定了“先用 TF 跑通深度学习基准，再用 MiniRocket 做特征提取对比”的复现策略。

(2) 内存溢出 (OOM) 与硬件限制

Prompt：“Colab RAM 崩溃了，报错显示 ResourceExhaustedError: Out of memory (OOM)。数据量太大，同时跑 5-Fold CV 时跑到第三折就特别卡，我该怎么修改代码让它不要一次性吃满显存？”

AI 提供：建议在每个 Fold 结束时，导入 gc 库手动清理 Keras 会话 (K.clear_session()) 并进行强制垃圾回收，提供了一长串内存管理代码。

学生贡献（关键决策）：我评估了 AI 的方案，认为引入复杂的内存管理库会增加代码的不确定性。考虑到限于当前电脑硬件与 Colab 免费额度限制，且本次复现的首要目的是为了测试代码的可用性与模型是否能跑通，我果断否决了 AI 的复杂方案。在实验结果与原论文基准结果对比下，我选择在代码层面继续缩小 batch_size 与 epochs，成功以最小的代码改动代价跨越了 OOM 障碍。

(3) 评估指标修正

Prompt：原代码无法直观输出每折的统计学指标，我应该如何实现报告每折准确率及均值±标准差。

AI 提供：增加一个用于记录每折最终评估结果的列表，并在循环结束后使

用 `numpy` 进行统计计算。

学生贡献：拒绝了 AI 仅“在循环后计算”的粗略建议，我重构了评估代码：在每折结束时评估并记录每一折的准确率，并在 `for` 循环完全结束后，导入 `numpy` 计算均值和标准差并格式化打印输出。

（4）模型解耦

Prompt：原代码将 `InceptionTime` 和 `ConvMHSA` 混写

AI 提供：`InceptionTime` 模型和 `ConvMHSA` 模型都有提供，此代码在训练循环中被注释掉了 `ConvMHSA` 的使用

学生贡献：将混写的代码拆分为两个独立的 `.py/.ipynb` 文件，分别控制变量进行训练。

2.4 实验与改进阶段

（1）分类任务模式的创新设计

Prompt：要实现维护事件二分类检测（维护前 vs 维护后航班），代码里应该使用哪个 `mode` 参数？

AI 提供：建议直接使用 `mode = 'before_after'`，这是最符合二分类字面意思的设置

学生贡献：可激活 `mode = 'both'`，在 `both` 模式下，模型不仅学习“维护前/后”，还会同时学习故障的具体分类，够提升模型提取特征的泛化能力，作为本次复现的一项合理改进。

三、认知迭代轨迹

3.1 从“语法报错处理”到“底层架构控制”

（1）**初始状态：**初期面对 `Python` 复杂的环境和数据结构，常常遇到 `TypeError`、`ReadError` 等基础报错，对模型训练流程缺乏全局观。

（2）**认知升级：**通过与 AI 的多轮追问，我不仅掌握了如何阅读错误堆栈，更理解了深度学习的训练生命周期。例如，关于“何时输出 `Mean Acc`”的疑惑：我起初以为 `Epoch` 跑完就会出结果，后来深刻理解了 **5-Fold Cross Validation** 的空间划分逻辑——必须等 5 个独立的 `Fold`（每个 `Fold` 200 个 `Epoch`）全部跑完，跳出大循环后，才能计算最终的交叉验证均值。

3.2 时间序列特征提取的底层逻辑剖析

(1) **追问 Prompt:** “在查阅 Benchmark 后我决定不用传统的 Isolation Forest。请给我讲讲 MiniRocket 算法相比 LSTM 的优势？它依赖的‘随机卷积核 (Random Convolutional Kernels)’到底是什么意思？难道不需要像传统 CNN 那样通过反向传播来更新权重吗？”

(2) **AI 解答:** 解释了 MiniRocket 使用海量固定参数的随机卷积核（长度、权重随机），直接提取时序数据的特征响应，因此完全不需要反向传播计算梯度。

(3) **学生吸收与应用:** 这彻底刷新了我对卷积神经网络必须依赖 Backpropagation 的固有认知。我深刻意识到，在处理高维多变量时序数据（如航空传感数据）时，免除反向传播的 MiniRocket 极其契合我当前面临的 Colab 硬件与内存瓶颈，这也是我最终选择它作为核心特征提取算法的根本原因。

四、 协作总结

在本次复现项目中，AI 是一位极其高效的“代码字典”和“概念导师”。但真正的核心价值产生于我对 AI 输出结果的怀疑、过滤与重构。无论是面对 OOM 报错时坚持“测试可用性优先”而放弃冗余内存代码，还是突破常规启用 `mode='both'` 进行多任务学习，都证明了在人机协同的闭环中，人类的业务理解、资源权衡与最终决策权是不可替代的。